

Problem Statement: Operation "Abhedya-Chakra"

Event: Void Hacks() 8.0

Theme: Abhedya – Cyber Security & Digital Forensics

In Association with: Indore Police Commissionerate

Dataset Scale: 2,000,000 Banking Records

Execution Window: 36 Hours

1. Problem Context & Law Enforcement Challenge

In financial cybercrimes (digital arrests, fake task schemes, Ponzi bots, and loan app frauds), criminals rapidly launder stolen money across multi-tiered **Money Mule Account Networks**. When the **Indore Police / 1930 Cyber Cell** investigates a major fraud syndicate, they receive multi-bank bulk transaction exports containing millions of entries.

Currently, detecting these rings fails due to three fundamental bottlenecks:

1. **Computational Scale:** Traditional spreadsheets crash on millions of rows. Standard relational queries take hours to calculate multi-hop transfers.
2. **Layering & Smurfing Obfuscation:** Organized syndicates deploy "Collector Mules" (Layer 1) that disperse funds into dozens of "Distributor Mules" (Layer 2) within minutes, before funneling them into "Terminal Cash-Out Nodes" (Layer 3: Crypto P2P, ATM withdrawers, or offshore payment gateways).
3. **Evidentiary Bottlenecks:** Officers need court-ready **Section 91 CrPC / BNSS Bank Freezing Notices** generated instantaneously to stop fund dissipation.

2. The Core Objective

Build a locally deployable, high-throughput analytics engine that:

1. Ingests and indexes **2,000,000+ transaction records** in under 60 seconds on standard developer laptops.
2. Applies graph topology algorithms to detect multi-tier money laundering patterns (pass-through velocity, fan-in/fan-out, cyclical smurfing).
3. Renders an interactive, sub-second visual transaction graph tracking the money trail from victim accounts down to terminal cash-outs.
4. Uses an integrated, prompt-injection-safe AI module to auto-generate a structured, legally sound **Police Case Diary and Bank Freeze Requisition**.

3. Detailed Functional Requirements

Module A: High-Throughput Ingestion & Normalization Engine

- **Big-Data Parsing:** System must stream/chunk and index 2 million records without encountering Out-Of-Memory (OOM) errors. Must handle memory footprint efficiently (using engines like DuckDB, Polars, SQLite, or Arrow).

- **Entity Disambiguation:** Extract and normalize bank account numbers, IFSC codes, payment rails (UPI, IMPS, NEFT, RTGS), IP anomalies (domestic vs. foreign proxy headers), and narration markers.
- **Instant Search & Querying:** Sub-second retrieval of any given account's complete historical flow, counter-parties, and transaction timeline.

Module B: Mule Ring Detection & Graph Analytics Heuristics

Implement graph-algorithmic detection to score and flag accounts on a 0–100 Mule Risk Index:

- **High-Velocity Pass-Through:** Flag nodes where $\geq 90\%$ of incoming funds are dispersed within 3 to 15 minutes across multiple outgoing transfers.
- **Fan-In / Fan-Out Topologies:**
 - *Collector Mules (L1):* High in-degree centrality, multiple distinct senders depositing money into a single node.
 - *Distributor Mules (L2):* High out-degree centrality, slicing incoming funds into 3–7 downstream accounts.
- **Terminal Cash-Out Identifiers (L3):** Outward transfers to payment wallets, P2P crypto narrations, foreign IP headers (185.x.x.x, 194.x.x.x), and headless script user-agents (Web_Emulator, Linux_Script).
- **Graph Multi-Hop Trace:** Given a Victim Account ID, compute the downstream money trail up to 4 hops deep within < 2 seconds.

Module C: Interactive Law Enforcement Flow Graph

- **Interactive Visualization:** Graph UI (React-Force-Graph, D3.js, Cytoscape, or Vis.js) capable of rendering clustered suspect subgraphs without freezing the browser.
- **Temporal Playback Slider:** Ability to filter transactions minute-by-minute across the 15-day timeline to show the exact propagation of stolen capital.
- **One-Click Subgraph Isolation:** Clicking a suspect account isolates its entire syndicate ring and exports the associated sub-dataset.

Module D: Local AI Case Officer & Legal Notice Generator

- **Automated FIR & Case Diary Generation:** Summarize the money trail narrative into a chronological police case diary:
 - Total funds siphoned from victim
 - Layer-wise accounts identified with timestamps and exact amounts
 - Current holding accounts recommended for immediate freezing
- **Automated Legal Freeze Requisition (Sec. 91 CrPC / BNSS Format):** Generate a pre-formatted, printable official notice addressed to Nodal Officers of respective banks (SBI, HDFC, ICICI, etc.) with exact beneficiary account numbers, IFSCs, and disputed transaction IDs.

- **Strict Anti-Hallucination Guardrail:** The AI engine must be programmatically prevented from hallucinating account numbers or amounts not present in the graph database.

4. Non-Functional & Performance Constraints

Requirement	Target Benchmark
Ingestion Benchmark	Full 2,000,000 rows loaded and indexed within ≤ 60 seconds on 16GB RAM hardware.
Multi-Hop Trace Latency	≤ 2 seconds to return a 4-hop money trail for any given victim account.
Zero Cloud Compute Reliance	Core data ingestion, graph computation, and dashboard rendering must execute offline/locally.
System Resilience	Web UI must remain responsive when rendering subgraphs containing 500+ nodes and 1,500+ edges.

5. Live Evaluation & Blind Testing (Jury Criteria)

During the 36-hour finale, judges (Police Officials & Industry Mentors) will test solutions as follows:

1. **The Blind Victim Query Test (40%):** Judges will provide 5 unannounced Victim Account IDs. The team's engine must trace the money trail, identify the exact Layer 1, Layer 2, and Layer 3 accounts, and display the flow on the graph in real-time.
2. **Detection Precision & Recall (30%):** Performance on identifying the 1,500 ground-truth injected mule accounts while minimizing false positives on 23,500 regular accounts.
3. **Court-Ready Output & Usability (20%):** Quality, factual accuracy, and readiness of the generated Section 91 notice and police diary.
4. **Architecture & Engineering Rigor (10%):** Clean codebase, memory management, and deployment simplicity.

Dataset Structure (11 Columns):

- **Transaction_ID:** Unique alphanumeric transaction identifier.
- **Sender_Account:** 12-digit originating account number.
- **Receiver_Account:** 12-digit beneficiary account number.
- **Sender_IFSC:** IFSC prefix mapping to 10 major Indian public/private banks.
- **Receiver_IFSC:** Destination IFSC code.
- **Amount:** Transaction value in INR (₹).
- **Timestamp:** Timestamp across a 15-day window (YYYY-MM-DD HH:MM:SS).
- **Payment_Mode:** UPI, IMPS, NEFT, or RTGS.

- Narration: Transaction remarks (includes authentic bank transfer codes and scam markers).
- IP_Address: Originating client IP (includes domestic residential IPs and foreign proxy anomalies).
- Device_Type: Client device profile (Android, iOS, Windows_Browser, Web_Emulator, Linux_Script).